

## Assignment-2

# Parallel Sorting Algorithms

Sorting is an essential operation used by a downstream application. GPUs, due to their massive parallelism, can play an essential role in realising faster versions of sorting algorithms.

We have listed some popular sorting algorithms that can be parallelised on GPUs.

| Sl No | Algorithm Name | Reference                                                                                                                                                                                                                                                                                                                                                                          |
|-------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.    | Radix Sort     | <a href="https://gpuopen.com/download/Introduction_to_GPU_Radix_Sort.pdf">https://gpuopen.com/download/Introduction_to_GPU_Radix_Sort.pdf</a>                                                                                                                                                                                                                                      |
| 2.    | Bitonic Sort   | <a href="https://www.geeksforgeeks.org/dsa/bitonic-sort/">https://www.geeksforgeeks.org/dsa/bitonic-sort/</a><br><a href="https://developer.nvidia.com/gpugems/gpugems2/part-vi-simulation-and-numerical-algorithms/chapter-46-improved-gpu-sorting">https://developer.nvidia.com/gpugems/gpugems2/part-vi-simulation-and-numerical-algorithms/chapter-46-improved-gpu-sorting</a> |
| 3.    | Sample Sort    | <a href="https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=5470444">https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=5470444</a>                                                                                                                                                                                                                                            |
| 4.    | Quick Sort     | <a href="https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&amp;arnumber=10094180">https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&amp;arnumber=10094180</a>                                                                                                                                                                                                                          |
| 5.    | Merge Sort     | <a href="https://en.wikipedia.org/wiki/Merge_sort#Parallel_merge_sort">https://en.wikipedia.org/wiki/Merge_sort#Parallel_merge_sort</a>                                                                                                                                                                                                                                            |

## The GPU Sorting Challenge

- Pick one algorithm of your choice.
- Implement it on the GPU and optimise it as much as possible to get the maximum performance. Also, try to conserve the amount of memory consumed.
- We will publish a leaderboard (updated daily) that captures the performance of each submitted entry.
- The top 3 implementations under each category will receive bonus marks.
- Note: The reference resources are given to help you quickly get bootstrapped with the implementation. You are free to modify. Optimise the algorithm as you like. All we care about is the correctness of the result and performance.

# Inputs

The list of input sequences is provided in CSV format. Each row corresponds to one input sequence. Each sequence need not be of the same length.

A sample Python script is provided for you to generate the input sequence for your testing. The script takes two input values:

- Max length of the sequence (L)
- Number of sequences (N)

Also, the sample input and output files are provided to illustrate the corresponding file/data formats. Your program should strictly accept inputs and generate the outputs in this format only.

## Directory Structure and Information

- The directory that you submit must follow the following format:
  1. The directory name must be your roll\_number.zip (in **UPPERCASE**)
  2. The directory should only contain the [main.cu](#) file.(The file name should be only [main.cu](#))
- The TAs should only be running the executable using the following command:  
./<executable\_name> <path\_to\_the\_input\_file>.csv -o <output\_file\_name>.csv
- The output file name should be <Input\_file\_name>\_output.csv If the output file name is incorrect, the autograder will assign **zero** for the corresponding test case.
- The compilation command that will be used to compile the programs is “nvcc [main.cu](#) -O3 -o main.out”
- After the end of each day (After 10 PM), your ranks (along with average execution times and Peak memory usages) will be available to you in the following format.
- Notice that the private test cases will not be provided until the end of this challenge.

| Author | Execution time | Peak Global and Shared | Category(Algo) | Rank |
|--------|----------------|------------------------|----------------|------|
|        |                |                        |                |      |
|        |                |                        |                |      |

# Evaluation

We will test your program with different L and N values for correctness and also measure the end-to-end time taken. The max value of L and N supplied will be 128 and 30K, respectively.

## Leaderboard logistics

- Private test cases will not be shared till the challenge ends. The test cases will remain fixed for the entire duration.
- The leaderboard will report the following for each category
  - Author
  - Execution time
  - Peak Memory usage: Global and shared
  - Rank
- The execution time will be the average of five runs
- Three TCs will be used N = 30000, L 128
  - Random
  - Semi-sorted
  - Sorted
- Leaderboard will be updated every day

## References

1. <https://cds.iisc.ac.in/wp-content/uploads/ParallelSorting.pdf>
2. <https://users.wfu.edu/choss/CUDA/docs/Lecture%2010.pdf>